

KubaMedia Innovations

Avto2W

Instrukcja Użytkownika
(2024-12-20)

Opracował: Jakub Chorąży
Sprawdził: Jakub Chorąży

KubaMedia Innovations
03-287, Warszawa
tel. 538 991 041

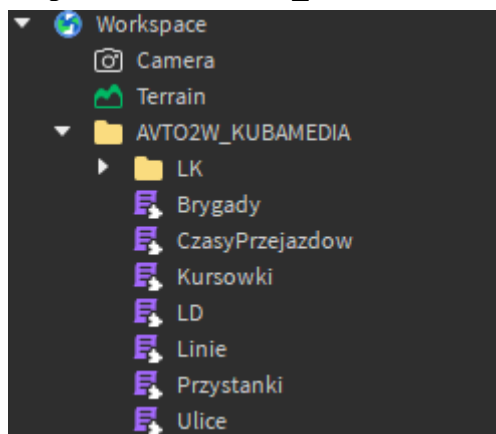
Podczas tworzenia poniższej instrukcji dołożono wszelkich starań aby dokładnie opisać krok po kroku proces tworzenia danych do sterowników firmy KubaMedia. W razie zastrzeżeń prosimy o kontakt mailowy z autorem dokumentu: jakub.chorazy35@gmail.com.

Spis treści

1.Opis programu.....	4
1.1.Opis modułów.....	4
2.Procedura tworzenia danych do sterownika.....	5
2.1.Tworzenie danych w “Spis ulic”.....	6
2.2.Tworzenie danych w “Spis przystanków”.....	7
2.3.Tworzenie danych w “Spis linii”.....	8
2.4.Tworzenie danych w “Spis kursówka”.....	9
2.5.Tworzenie danych w “Spis brygady”.....	11
3.Tworzenie napisów dodatkowych dla sterownika.....	12
4.Wgrywanie komunikatów o numerze linii i kierunku.....	13

1. Opis programu

Program to inaczej moduły skryptowe, które zawierają tabele pozwalające przygotować bazę danych do sterownika SKML-1 oraz SKML-1v2. Moduły skryptowe powinny być umieszczone w ścieżce **Workspace -> AVTO2W_KUBAMEDIA**



1.1. Opis modułów

W podstawowej wersji programu posiadamy następujące moduły skryptowe:

Brygady	Służący do przypisania brygad do danej linii
Kursowki	W tym module skryptowym tworzy się cały przebieg danej trasy podzielonej na różne warianty
LD	Spis napisów dodatkowych
Linie	Spis wszystkich linii, wraz z kierunkami jakie mają zostać wyświetlone na tablicach
Przystanki	Spis wszystkich przystanków, zawiera m.in. Numer słupka, numer gps, nazwę przystanku, przypisanie ulicy do przystanku, strefę przystankową oraz asset id komunikatu
Ulice	Spis wszystkich ulic, które będą następnie przypisane do danego przystanku

2. Procedura tworzenia danych do sterownika

Poniższe kroki przedstawiono w odpowiedniej kolejności, tworzenie bazy w innej kolejności może skutkować złym odczytem danych ze sterownika lub brakiem danych.

Spis ulic	Na samym początku tworzymy spis ulic, aby mieć co przypisać w spisie przystanków
Spis przystanków	Tworzymy spis przystanków, które mają znaleźć się na trasach linii utworzonych w następnym kroku
Spis linii	Następnym krokiem jest utworzenie spisu linii, w spisie linii dodajemy takie informacje jak numer linii oraz kierunek wyświetlany na tablicach kierunkowych
Spis kursówki	W tym spisie należy utworzyć odpowiedni przebieg trasy wykorzystując dane zawarte w spisie przystanków odwołując się po ID z tabeli przystanków
Brygady	Ostatnią czynnością jest przypisanie brygad dla każdej linii, każda linia musi mieć minimum jedną brygadę aby została wyświetlona w sterowniku

W dalszej części instrukcji każdy krok z procedury został bardziej dokładnie opisany. Zaleca się zapoznanie z całą lekturą.

2.1. Tworzenie danych w “Spis ulic”

Aby rozpocząć tworzenie danych w module skryptowym o nazwie “Ulice”, należy dwukrotnie wcisnąć LPM. Następnie otwiera nam się poniższa tabela

```
1  --[[
2
3
4  SOLITONS BY
5
6  KUBA MEDIA
7
8
9
10
11
12
13
14  All rights reserved. Kubamedia, author discord: slonis.
15  ]]--
16
17  local Street = {
18      {id = 1, nazwa = "Moniuszki" },
19      {id = 2, nazwa = "Długa" },
20      {id = 3, nazwa = "Noworudzka" },
21      {id = 4, nazwa = "Powstańców Śląskich" },
22      {id = 5, nazwa = "Gen. Ziętka" },
23      {id = 6, nazwa = "Wojska Polskiego" },
24      {id = 7, nazwa = "Górnicza" },
25      {id = 8, nazwa = "Osiedlowa" },
26
27
28
29
30  }
31
32  return Street
33
```

Budowa tabeli:

{id = **n**, nazwa = “Nazwa przystanku” },

gdzie, n to liczba porządkowa w spisie ulic należy przypisywać je po kolei.

Każdy rekord musi być umieszczony w {} oraz po ostatniej klamercie należy umieścić znak przecinka. W polu nazwa wprowadzamy nazwę ulicy, musi ona zostać podana w cudzysłowie.

2.2. Tworzenie danych w “Spis przystanków”

Aby rozpocząć tworzenie danych w module skryptowym o nazwie “Przystanki”, należy dwukrotnie wcisnąć LPM. Następnie otwiera nam się poniższa tabela

```
1  --[[
2  SOLUTIONS BY:
3  KUBAMEDIA
4
5
6
7
8
9
10
11
12
13
14  All rights reserved. Kubamedia, author discord: slonis.
15  ]]]--
16
17  local street = require(script.Parent.Ulice)
18
19  local Busstop = {
20    {id = 01, numer_gps = 1, nazwa = "Wielka Skotnica", id_ulicy = street[1].id, nazwa_ulicy = street[1].nazwa, strefa_prz = 50, asset_id = "9031"},
21    {id = 01, numer_gps = 2, nazwa = "Targowa", id_ulicy = street[1].id, nazwa_ulicy = street[1].nazwa, strefa_prz = 50, asset_id = "124995858406"},
22    {id = 01, numer_gps = 3, nazwa = "Rondo Kopydło", id_ulicy = street[2].id, nazwa_ulicy = street[2].nazwa, strefa_prz = 50, asset_id = "800151"},
23    {id = 01, numer_gps = 4, nazwa = "Zajezdnia Kołobrzęska", id_ulicy = street[2].id, nazwa_ulicy = street[2].nazwa, strefa_prz = 50, asset_id = "800151"},
24    {id = 01, numer_gps = 5, nazwa = "Kołobrzęska-Szkoła", id_ulicy = street[2].id, nazwa_ulicy = street[2].nazwa, strefa_prz = 50, asset_id = "800151"},
25    {id = 01, numer_gps = 6, nazwa = "Nadwiślańska", id_ulicy = street[3].id, nazwa_ulicy = street[3].nazwa, strefa_prz = 50, asset_id = "9862693"},
26    {id = 01, numer_gps = 7, nazwa = "Nowa Ruda-Urząd Stanu Cywilnego", id_ulicy = street[4].id, nazwa_ulicy = street[4].nazwa, strefa_prz = 50, asset_id = "9862693"},
27    {id = 01, numer_gps = 8, nazwa = "Bohaterów Monte Cassino", id_ulicy = street[4].id, nazwa_ulicy = street[4].nazwa, strefa_prz = 50, asset_id = "9862693"},
28    {id = 01, numer_gps = 9, nazwa = "Śląska", id_ulicy = street[4].id, nazwa_ulicy = street[4].nazwa, strefa_prz = 50, asset_id = "1398905261088"},
29    {id = 01, numer_gps = 10, nazwa = "Sęczkowice-Kościół", id_ulicy = street[5].id, nazwa_ulicy = street[5].nazwa, strefa_prz = 50, asset_id = "1398905261088"},
30    {id = 01, numer_gps = 11, nazwa = "Sęczkowice-Dworcowa", id_ulicy = street[5].id, nazwa_ulicy = street[5].nazwa, strefa_prz = 50, asset_id = "1398905261088"},
31    {id = 01, numer_gps = 12, nazwa = "Wojska Polskiego", id_ulicy = street[6].id, nazwa_ulicy = street[6].nazwa, strefa_prz = 50, asset_id = "76"},
32    {id = 02, numer_gps = 13, nazwa = "Kopalnia Dziechcinka", id_ulicy = street[7].id, nazwa_ulicy = street[7].nazwa, strefa_prz = 50, asset_id = "76"},
33    {id = 02, numer_gps = 14, nazwa = "Osiedlowa", id_ulicy = street[8].id, nazwa_ulicy = street[8].nazwa, strefa_prz = 50, asset_id = ""},
34  }
35
36
37
38  return Busstop
39
```

Konstrukcja tabeli jest następująca:

{id = xx, numer_gps = yy, nazwa = “Nazwa przystanku”, id_ulicy = street[n], nazwa_ulicy = street[n], strefa_prz = 50, asset_id = “k”},

gdzie:

xx - Numer przystanku np. Dworzec Wileński **07**

yy - Numer dla modułu UGM oraz Impuls lub GPS do wykrycia danego przystanku. Należy numerować najlepiej w kolejności numerycznej

n - numer id ze spisu ulic, dlatego w spisie ulic powinno być to numerowane po kolei ponieważ jeśli id się zaczyna np. od 2300 to sterownik może nie wyszukać rekordu danych

k - sam numer asset id dźwięku samej wymowy nazwy przystanku, moduł UGM sam skleja komunikaty dodając odpowiednie przed i po. Należy dodać wymowę tylko nazwy przystanku

2.3. Tworzenie danych w “Spis linii”

Aby rozpocząć tworzenie danych w module skryptowym o nazwie “Linie”, należy dwukrotnie wcisnąć LPM. Następnie otwiera nam się poniższa tabela

```
1  --[[
2
3
4  SOLUTIONS BY:
5
6  KUBAMEDIA
7
8
9
10
11
12
13
14  All rights reserved. Kubamedia, author discord: slonis.
15  ]]--
16
17  local Linie = {
18    {id = 1, linia = "50", kierunek = {"Kopalnia\\Dziehcinka", "Wielka\\Skotnica", "Osiedlowa<f3>"},
19  }
20  }
21
22  return Linie
23
```

Konstrukcja tabeli jest następująca:

{id = n, linia = “Numer linii”, kierunek = {“Wariant1”, “Wariant2”}},

gdzie:

n - Liczba porządkowa w tabeli, należy podawać w kolejności numerycznej

W powyższym spisie rekord o nazwie “linia” zawiera numer linii, który będzie zawarty np. w sterowniku oraz tablicach kierunkowych. W zależności od software’u sterownika będą obsługiwane numery linii zawierające litery i inne znaki specjalne. Następnie w rekordzie o nazwie “kierunek” znajdują się kierunki przypisane dla każdego wariantu, warianty liczone są od lewej do prawej i mogą mieć nieograniczoną liczbę. Aby rozdzielić treść na tablicach należy w wybranym miejscu dodać \ .

Tablice z odpowiednim softem obsługują również tagi <fx>, gdzie x to numer trybu specjalnego np. Trasa zmieniona. Tagi te nie będą tutaj wypisywane ponieważ mogą różnić się w zależności od oprogramowania tablicy i życzeń klienta.

2.4. Tworzenie danych w “Spis kursówka”

Aby rozpocząć tworzenie danych w module skryptowym o nazwie “Kursowki”, należy dwukrotnie wcisnąć LPM. Następnie otwiera nam się poniższa tabela

```
1  --[[
2
3
4
5
6
7
8
9
10
11
12
13
14  All rights reserved. Kubamedia, author discord: slonis.
15  ]]--
16
17  local stop = require(script.Parent.Przystanki)
18
19  local kursowka = {
20    ['5001'] = {
21      P1 = stop[1],
22      P2 = stop[2],
23      P3 = stop[3],
24      P4 = stop[4],
25      P5 = stop[5],
26      P6 = stop[6],
27      P7 = stop[7],
28      P8 = stop[8],
29      P9 = stop[9],
30      P10 = stop[10],
31      P11 = stop[11],
32      P12 = stop[12],
33      P13 = stop[13],
34      P1NZ = false,
35      P2NZ = true,
36      P3NZ = true,
37      P4NZ = false,
38      P5NZ = false,
39      P6NZ = true,
40      P7NZ = false,
41      P8NZ = false,
42      P9NZ = false,
43      P10NZ = false,
44      P11NZ = true,
45      P12NZ = false,
46      P13NZ = false,
47    },
48  },
49
50  }
51
52  return kursowka
53
```

Konstrukcja tabeli jest następująca:

[‘numerlinii...numerwariantu’] = {

P1 = stop[n],
P2 = stop[n],
.....
P1NZ = false/true,
P2NZ = false/true,
.....

},

gdzie:

n - Numer GPS z tabeli przystanków

W powyższej tabeli można ustawić również czy dany przystanek ma być warunkowy czy stały poprzez oznaczenie go jako false lub true. True będzie oznaczało, że przystanek będzie warunkowy a false - stały.

Budowanie danych rozpoczynamy od ustalenia numeru linii a następnie numeru wariantu w nawiasie kwadratowym [] na samej górze tabeli. Przykładowo dla linii 120 wariant 13 będzie nawias kwadratowy wyglądał tak - 12013. W przypadku nowszego software'u sterownika, gdzie można wprowadzać numery linii złożone z liter, dla linii N11 wariant 13 nawias kwadratowy będzie wyglądał tak - N1113. Ważne jest aby przystanki odpowiednio ponumerować w kolejności numerycznej, chodzi o tagowanie przystanków np.:

$P_x = \text{stop}[n]$,

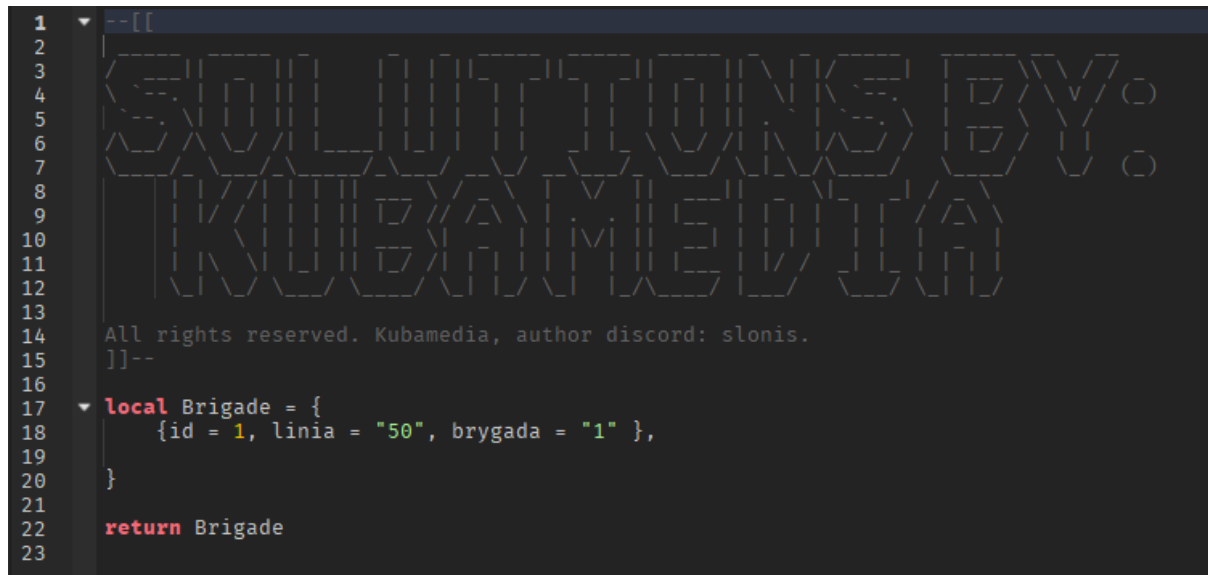
wszystkie przystanki muszą być ponumerowane po kolei np. dla przystanku z spisu przystanków o numerze gps równym 12 tag będzie wyglądał następująco

$P_1 = \text{stop}[12]$,

Zawsze na sam koniec rekordu należy dodać przecinek umożliwiający wprowadzanie dalej danych. Podobna sytuacja będzie wyglądać dla oznaczeń czy przystanek jest na żądanie czy jest stały. Oznaczeń stały/warunek musi być tyle samo co liczba przystanków w tabeli, a więc jeśli mamy od P1 do P13 to taka sama musi być ilość P_xNZ , muszą być wypisane one po kolei aby zachować porządek w kolejności.

2.5. Tworzenie danych w “Spis brygady”

Aby rozpocząć tworzenie danych w module skryptowym o nazwie “Brygady”, należy dwukrotnie wcisnąć LPM. Następnie otwiera nam się poniższa tabela



```
1  --[[
2
3
4  SOLUTIONS BY:
5
6  KUBAMEDIA
7
8
9
10
11
12
13  All rights reserved. Kubamedia, author discord: slonis.
14  ]]--
15
16
17  local Brigade = {
18      {id = 1, linia = "50", brygada = "1" },
19  }
20
21
22  return Brigade
23
```

Konstrukcja powyższej tabeli jest następująca:

{id = n, linia = “Numerlinii”, brygada = “Numerbrygady”}.

gdzie:

n = Liczba porządkowa w tabeli, należy nadawać ją w kolejności numerycznej

W rekordzie o nazwie “linia” wprowadzamy numer zgodny z tym co zostało podane w spisie linii. W zależności od softu, jeżeli sterownik obsługuje numery linii z literami, w tym rekordzie należy również wprowadzić numer linii z literą np. N11. W rekordzie o nazwie “brygada” wprowadzamy numer brygady, który będzie wyświetlał się w sterowniku i na tablicy brygadowej. Również w zależności od softu można wprowadzić dane z literą np. M13.

3. Tworzenie napisów dodatkowych dla sterownika

Aby rozpocząć tworzenie danych w module skryptowym o nazwie “LD”, należy dwukrotnie wcisnąć LPM. Następnie otwiera nam się poniższa tabela

```
1  --[[
2
3
4  SOLUTIONS BY:
5
6
7
8
9
10
11
12
13
14  All rights reserved. Kubamedia, author discord: slonis.
15  ]]--
16
17  local LD = {
18    {id = 1, nazwa = "PRZEJAZD\\TECHNICZNY" },
19    {id = 2, nazwa = "Pierwsza linia tramwajowa w\\Nowej Rudzie!" },
20    {id = 3, nazwa = "MZK NOWA RUDA\\ŻYCZY MIŁEGO DNIA :)"} ,
21
22
23
24  }
25
26  return LD
27
```

Konstrukcja tabeli jest następująca:

{id = n, nazwa = “Własny tekst”},

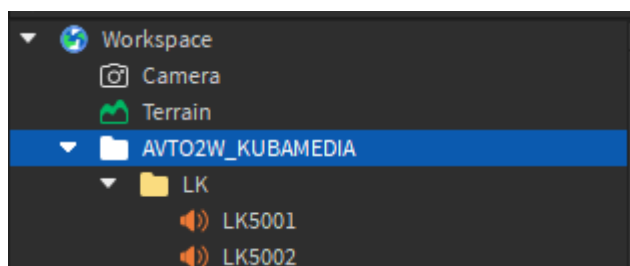
gdzie:

n - Liczba porządkowa, należy ją podawać w kolejności numerycznej

Tworzenie napisów własnych jest bardzo prostą czynnością, tworzymy tyle rekordów ile nam potrzeba i odpowiednio je wypełniamy. W rekordzie o nazwie “nazwa” wprowadzamy dowolny tekst, który może być podzielony na dwie linie bądź wyświetlony w jednej linii. Podział na dwie linie następuje po dodaniu \\.

4. Wgrywanie komunikatów o numerze linii i kierunku

Wgrywanie komunikatów o numerze linii i kierunku, które mogą być wygłaszane przez moduł UGM, wrzuca się jako pliki dźwiękowe (soundy) do folderu LK w głównym folderze AVTOW2_KUBAMEDIA. Poniżej zawarto strukturę plików



Nazewnictwo plików dźwiękowych:

LK - Prefix

50 - Numer linii

01 - Numer wariantu

Przykładowo komunikat dla linii 123 o wariancie 12 będzie wyglądać następująco

LK12312

Prefix LK musi być zawarty zawsze na początku pliku!

Komunikaty są dostępne w zależności od software'u modułu głośnomówiącego oraz sterownika.